

INTRODUCTION TO

Reinforcement Learning

Fundamentals · Components · Model Types

CSE4020 — Capstone Design | Dr. Oybek Eraliev

Table of Contents

What we will cover in this lecture

- 01** What is Reinforcement Learning?
- 02** Why RL? Motivation & Real-World Applications
- 03** RL Fundamentals: Key Concepts
- 04** Components of an RL System
- 05** The RL Loop & Markov Decision Processes

What is Reinforcement Learning?

Section 01 — Definition & Intuition

"Reinforcement Learning is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative reward."



Learning by Doing

Agent learns from trial and error,
not labeled data



Goal-Directed

Optimizes towards a long-term
objective, not instant accuracy



Sequential Decisions

Choices are made over time; each
action affects the future

What is Reinforcement Learning?

Section 01 — Definition & Intuition



Learning by Doing

Example: A Robot Vacuum. It doesn't have a map of a new room. It bumps into a wall (negative reward), turns, and finds a dusty patch (positive reward). It "learns by doing" which movements lead to cleaner floors without being told the coordinates of every piece of furniture.



Goal-Directed

Example: Chess. An agent might choose to "sacrifice" its Queen (a massive immediate loss/low reward) because its internal model predicts that this specific move leads to a Checkmate (the ultimate goal) five moves later.



Sequential Decisions

Example: Self-Driving Cars. Driving isn't a one-shot "Stop or Go" decision. It is a sequence: you must first signal, then check the mirror, then merge, then accelerate. If you don't signal (Action 1), the state of the traffic changes (other cars honk or block you), constraining your next possible actions.

RL vs. Supervised & Unsupervised Learning

How RL differs from other ML paradigms

Supervised Learning	Unsupervised Learning	Reinforcement Learning
● Learns from labeled data ● Teacher provides correct answers ● Static dataset ● Predict output from input ● Examples: image classification, regression	● No labels provided ● Finds hidden patterns ● Clustering, dimensionality reduction ● No explicit feedback ● Examples: Autoencoders, k-mens	● Learns from interaction ● Reward signal — not labels ● Dynamic environment ● Sequential decision-making ● Examples: game playing, robotics

Why RL? Motivation & Applications

Section 02 — Motivation

Going beyond supervised (x, y) examples

- AI model predictions have consequences
- When direct supervision isn't available

Widely used and deployed for performant AI systems

Learning from experience seems fundamental to intelligent

- RL can discover new solutions

Plenty of existing open research problems

Why RL? Motivation & Applications

Section 02 — Real-world impact



Game Playing

AlphaGo, OpenAI Five, Atari DQN
— superhuman performance



Robotics

Robot manipulation, locomotion,
autonomous navigation.

<https://www.youtube.com/watch?v=WADmP0wzLxs>



Autonomous Driving

Lane changes, intersection
decisions, path planning



Healthcare

Treatment planning, drug
discovery, clinical trials



Finance

Algorithmic trading, portfolio
optimization



Energy Systems

Smart grid control, data center
cooling (Google DeepMind)

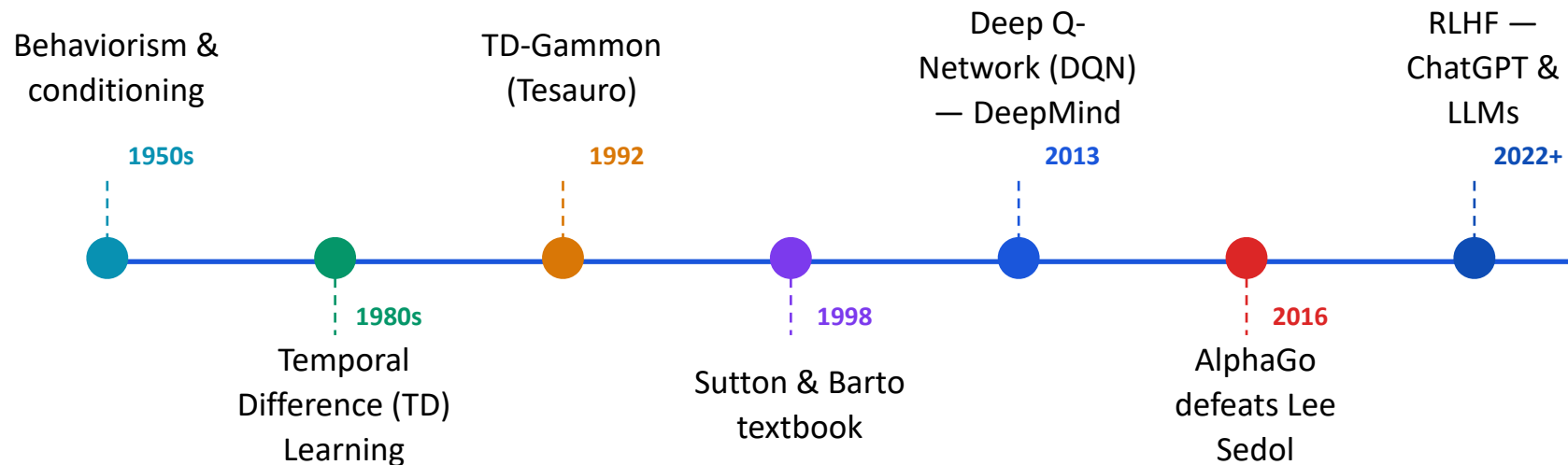
Why RL? Motivation & Applications

Section 02

- Recommendation to watch
- <https://www.youtube.com/watch?v=d95J8yzvjbQ&t=4880s>

A Brief History of Reinforcement Learning

From behaviorism to deep RL breakthroughs



A Brief History of Reinforcement Learning

From behaviorism to deep RL breakthroughs

1950s: Behaviorism & conditioning

- **The Concept:** Before RL was a computer science discipline, it was a psychological one. Researchers like B.F. Skinner studied how animals (and humans) learn through "operant conditioning"—repeating actions that yield positive outcomes and avoiding those that result in negative ones.
- **The Significance:** This is the biological and philosophical foundation of the "Learning by Doing" property. The fundamental mechanism of an agent interacting with an environment and receiving a reward signal stems directly from this era.

1980s: Temporal Difference (TD) Learning

- **The Concept:** Pioneered largely by Richard Sutton, TD learning introduced a mathematical way for an agent to learn *during* an experience, rather than waiting until the very end of an episode to update its knowledge. It learns by comparing its current prediction of future reward with its previous prediction.
- **The Significance:** This solved the problem of continuous learning. It became the core mathematical engine for systems that require continuous control and real-time path correction, laying the groundwork for complex tasks like autonomous navigation and robotic movement mapping.

A Brief History of Reinforcement Learning

From behaviorism to deep RL breakthroughs

1992s: TD-Gammon (Tesauro)

- **The Concept:** Developed by IBM researcher Gerald Tesauro, TD-Gammon was a computer program that learned to play backgammon at a world-championship level. It did this by playing over a million games against itself using TD learning combined with an artificial neural network.
- **The Significance:** This was the first major, highly visible success of combining RL with neural networks (function approximation). It proved that RL agents could discover strategies that human experts hadn't even considered.

1998: Sutton & Barto Textbook

- **The Concept:** Richard Sutton and Andrew Barto published the first edition of *Reinforcement Learning: An Introduction*.
- **The Significance:** This book unified the scattered research into a cohesive, formal academic discipline. It standardized the language we use today—Markov Decision Processes (MDPs), Q-learning, SARSA, and the Bellman equations—serving as the definitive foundation for university curriculum and capstone engineering projects worldwide.

A Brief History of Reinforcement Learning

From behaviorism to deep RL breakthroughs

2013: Deep Q-Network (DQN) — DeepMind

- **The Concept:** Researchers at DeepMind created an agent that could learn to play classic Atari 2600 games directly from the raw pixel data on the screen, without any hand-crafted rules or state definitions. They achieved this by merging Deep Learning (Convolutional Neural Networks) with Q-learning.
- **The Significance:** This birthed the modern era of **Deep RL**. By proving that neural networks could successfully process high-dimensional sensory inputs (like vision) into successful action policies, it opened the door for AI to tackle visually complex, real-world environments.

2016: AlphaGo Defeats Lee Sedol

- **The Concept:** DeepMind's AlphaGo system defeated Lee Sedol, an 18-time world champion in the ancient board game Go.
- **The Significance:** Go has a state space so massive (10^{170} possible board configurations) that traditional brute-force computing could never solve it. AlphaGo succeeded by combining Deep RL with Monte Carlo Tree Search to intuitively "evaluate" board positions. This was widely considered a landmark moment in artificial intelligence, occurring a decade earlier than experts predicted.

A Brief History of Reinforcement Learning

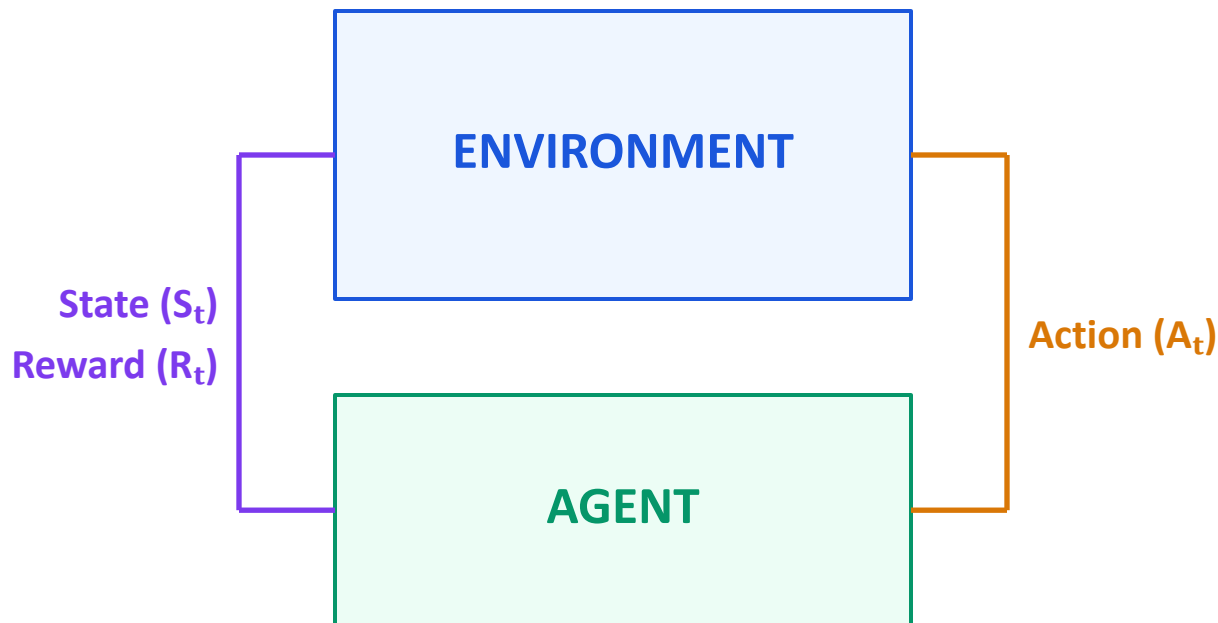
From behaviorism to deep RL breakthroughs

2022+: RLHF — ChatGPT & LLMs

- **The Concept:** Reinforcement Learning from Human Feedback (RLHF). Instead of using a hard-coded reward function (like points in a video game), a separate "Reward Model" is trained on thousands of human preferences. This reward model is then used to score and optimize the outputs of massive generative text models.
- **The Significance:** This is the breakthrough that transformed foundational language models into safe, highly steerable conversational agents. It bridged the gap between predicting the next token and actually following complex human instructions, which is critical when adapting models to highly specific, localized domains.

The Reinforcement Learning Loop

Section 03 — Fundamentals



Component: The Agent

Section 04 — Components of an RL System

The Agent is the learner and decision-maker. It observes states, selects actions, receives rewards, and updates its policy over time.

Policy (π)

The strategy or mapping from states to actions. Can be deterministic ($\pi(s) = a$) or stochastic ($\pi(a | s)$). This is what the agent learns.

Value Function (V or Q)

Estimates how good a state or state-action pair is. $V(s)$ = expected return from state s . $Q(s,a)$ = expected return from taking action a in state s .

Model (optional)

An internal representation of the environment. Used in model-based RL to plan ahead. Not all agents have a model.

Component: The Environment

Section 04 — Components of an RL System

The Environment is everything the agent interacts with. It receives actions and responds with new states and reward signals.

State Space (S)

All possible configurations of the environment. Can be discrete (grid world) or continuous (robot joint angles).

Transition Dynamics

$P(s' | s, a)$ — probability of moving to state s' from state s after taking action a . May be known or unknown.

Observation vs State

Fully observable: agent sees full state. Partially observable: agent only sees an observation of the state.

Reward Function $R(s,a)$

Defines what the agent should maximize. A scalar signal received after each step. Good reward design is crucial!

Reward, Return & Discount Factor

Section 03 — Fundamentals

The agent's goal is to maximize the expected cumulative reward over time — not just the immediate reward.

Reward at time t : $R_t \rightarrow$ scalar feedback signal from environment

Return (undiscounted): $G_t = R_t + R_{t+1} + R_{t+2} + \dots + R_T$

Return (discounted): $G_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \dots$ where $0 \leq \gamma \leq 1$

Discount Factor γ : $\gamma \rightarrow 1$ means far-future rewards matter equally. $\gamma \rightarrow 0$ means the agent is myopic (short-sighted). Typically $\gamma = 0.9$ to 0.99 .

Policy: The Agent's Strategy

Section 03 — Fundamentals

A policy π defines the agent's behavior — it maps each state to an action (or action probability distribution).

Deterministic Policy

$$\pi(s) = a$$

Directly maps each state to one specific action.
Simple and fast, but not always optimal in uncertain environments.

Stochastic Policy

$$\pi(a \mid s) = P[A = a \mid S = s]$$

Outputs a probability distribution over actions.
Allows exploration and handles partial observability better.

Goal: Find the optimal policy π^* that maximizes the expected return: $\pi^* = \operatorname{argmax}_{\pi} E[G_t \mid \pi]$

Value Functions: $V(s)$ and $Q(s, a)$

Section 03 — Fundamentals

Value functions estimate how good it is to be in a given state or to take an action, assuming a specific policy.

State-Value Function $V^\pi(s)$

$$V^\pi(s) = E_\pi[G_t \mid S_t = s]$$

Expected return starting from state s , following policy π . Tells the agent: 'How good is being in this state?'

Action-Value Function $Q^\pi(s, a)$

$$Q^\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$$

Expected return starting from state s , taking action a , then following policy π . Tells: 'How good is this action here?'

Bellman Equation: $V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$

Exploration vs. Exploitation Dilemma

One of the central challenges in RL

The agent must balance exploiting known good actions vs. exploring new actions that might be better.



Exploitation

- Use the best-known action
- Maximizes short-term reward
- Risk: miss better options
- Greedy strategy: pick $\text{argmax}_a Q(s,a)$



Exploration

- Try new / less-known actions
- May find better long-term policy
- Risk: temporary lower reward
- Random/epsilon-greedy sampling

ϵ -Greedy Strategy: With probability ϵ , take a random action (explore). With probability $1-\epsilon$, take the best known action (exploit). Typically ϵ starts high and decays.

Markov Decision Processes (MDPs)

Section 05 — The Mathematical Framework

An MDP provides the formal framework for RL problems. Almost every RL problem can be cast as an MDP.

MDP is defined by the tuple: (S, A, P, R, γ)

S	State Space	Set of all possible states the environment can be in
A	Action Space	Set of all actions the agent can take
P	Transition Function	$P(s' s, a)$ — probability of transitioning to s' given s and a
R	Reward Function	$R(s, a, s')$ — immediate reward received after transition
γ	Discount Factor	$0 \leq \gamma \leq 1$ — weights importance of future vs. present rewards

The Markov Property

Section 05 — Fundamentals

"The future is independent of the past given the present state."

$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1, S_2, \dots, S_t]$$

What this means

The current state S_t captures ALL relevant information. You don't need the full history of states to predict the next state.

Fully Observable (MDP)

The agent sees the complete state. The Markov property holds. Most basic RL algorithms assume this.

Partially Observable (POMDP)

The agent only sees a partial observation O_t , not the full state S_t . Requires belief states or recurrent networks (e.g., LSTM).

Why it matters

The Markov property simplifies the mathematics enormously — we only need to track the current state, not the entire trajectory.

Thank You!

Introduction to Reinforcement Learning

Coming Up:

- Q-Learning
- Implement Q-Learning from scratch
- Train your first PPO agent

Questions?

Dr. Oybek Eraliev
CSE4020 — Capstone Design